



Auditing Report

Hardening Blockchain Security with Formal Methods

FOR



Volo Vault



Veridise Inc.
March 25, 2026

► **Prepared For:**

Navi Protocol
www.volosui.com

► **Prepared By:**

Aayushman Thapa Magar
Evgeniy Shishkin
Mark Anthony

► **Contact Us:**

contact@veridise.com

► **Version History:**

Jul. 27, 2026	V3
Jul. 21, 2026	V2
Apr. 9, 2026	V1
Apr. 7, 2026	Initial Draft

Contents

Contents	iii
1 Executive Summary	1
2 Project Dashboard	5
3 Security Assessment Goals and Scope	6
3.1 Security Assessment Methodology	6
3.2 Identified Security Risks	6
3.3 Scope	7
3.4 Classification of Vulnerabilities	8
4 Security Review Assumptions	9
4.1 Operational Assumptions	9
4.2 Privileged Roles	9
5 Vulnerability Report	11
5.1 Detailed Description of Issues	12
5.1.1 V-VVT-VUL-001: Curator position value can be written to an arbitrary asset slot	12
5.1.2 V-VVT-VUL-002: Operator losses can be hidden through partial NAVI market updates	14
5.1.3 V-VVT-VUL-003: Withdraw with swap operation does not implement real slippage protection	15
5.1.4 V-VVT-VUL-004: Curator loop flows leave principal asset value stale . .	17
5.1.5 V-VVT-VUL-005: Oracle accepts future-dated prices as valid	19
5.1.6 V-VVT-VUL-006: NAVI valuation ignores liquidation risk	20
5.1.7 V-VVT-VUL-007: Unsupported NAVI reserve can prevent market valuation	21
5.1.8 V-VVT-VUL-008: Withdraw with swap requests can be executed as normal withdrawals	22
5.1.9 V-VVT-VUL-009: Vault deposit cap can be exceeded due to buffered pending deposits	24
5.1.10 V-VVT-VUL-010: Unsupported target asset can create an unexecutable withdraw-with-swap request	25
5.1.11 V-VVT-VUL-011: Slippage bps greater than rate scaling can brick zap-out execution	26
5.1.12 V-VVT-VUL-012: Withdraw minimum amount check does not protect net amount received	27
5.1.13 V-VVT-VUL-013: Maintainability concerns	28
A Appendix	29
A.1 Intended Behavior: Non-Issues of Note	29
A.1.1 V-VVT-APP-VUL-001: Adding a NAVI market leaves a stale position value	29
A.1.2 V-VVT-APP-VUL-002: Deposit cancellation routes funds to different recipients depending on the entry point	31
A.1.3 V-VVT-APP-VUL-003: Locking periods are mutable and can unfairly delay user exits	33

From Mar. 25, 2026 to Mar. 31, 2026, Navi Protocol engaged Veridise to conduct a security assessment of Volo Vault version 1.3, an upgrade to the previously reviewed Volo Vault. Veridise conducted the assessment over 15 person-days, with 3 security analysts reviewing the project over 5 days on commit f8b817d. Compared to the previous version, which Veridise reviewed previously*, this new version introduces a new type of withdrawal request.

In addition to the assessment described above, Veridise conducted a separate security review of [PR #27](#), which introduces multi-market NAVI support. The pull request was reviewed by 3 security analysts over 1 day on commit b151492

Project Summary. The **Volo Vault** is a yield-generating system implemented in the form of a vault. Users deposit their coins into the Vault and receive Vault shares in return. These shares can later be redeemed for their corresponding value — ideally higher than the initial purchase price — at a future time.

Yield within the Volo Vault protocol is expected to be generated by issuing flash loans from the deposited funds to a specifically designated set of users, known as *Operators*. These Operators utilize the borrowed assets to execute custom trading strategies across various DeFi protocols, such as *Navi*, *SuiLend*, *Momentum*, and others, thereby producing profits for the protocol.

The Volo Vault protocol operates on the **Sui blockchain** and follows a semi-centralized design. Part of the protocol is implemented through on-chain smart contract logic, while other components — notably the Operators' trading logic — are executed off-chain.

The typical operational workflow of the protocol is structured as follows:

1. **Deposits.** Users submit deposit requests to the Vault. In response, the Vault creates a *Receipt* object that uniquely identifies the pending deposit. The request is not fulfilled immediately; instead, it is added to a queue of pending deposit requests awaiting approval by an Operator.
2. **Deposit Approval.** At a chosen point, an Operator decides to process a pending deposit request. They explicitly execute this operation on-chain, specifying the unique identifier of the deposit to be fulfilled. Upon completion, the user receives the corresponding amount of Vault shares, while the deposited coins are allocated to a special *free principal balance*, making them available for flash loans.
3. **Yield Generation.** To generate yield, Operators implement custom trading strategies that temporarily borrow accumulated Vault funds via flash loans to interact with other DeFi protocols on Sui. These loans are executed within a single transaction using the *Programmable Transaction Blocks*(PTB)[†] of Sui. The protocol enforces a rule ensuring that the amount of assets returned is no less than the amount borrowed[‡]. Ideally, Operators return an amount greater than what was borrowed, thereby increasing the value of each Vault share.

* The previous audit report, if it is publicly available, can be found on Veridise's website at <https://veridise.com/audits-archive/>

[†] [Sui: Transaction Blocks Documentation](#)

[‡] An epoch loss mechanism allows for minor losses to accumulate if necessary.

4. **Withdrawals.** To redeem shares, users submit a withdrawal request, supplying the *Receipt* created during the deposit. Similar to deposits, withdrawal requests are not executed immediately but are placed into a queue of pending withdrawal requests overseen by Operators.
5. **Withdrawal Approval.** At a selected time, an Operator decides to process a pending withdrawal request. They execute this operation on-chain, specifying the unique identifier of the withdrawal to be fulfilled. Upon completion, the user receives their coins, an amount exceeding the initial deposit in an ideal scenario, reflecting the yield generated over time.

Compared to the previously reviewed version of **Volo Vault**, this release introduces the following changes:

- ▶ **Withdraw-and-swap functionality:** Users can now request withdrawals in a token different from the Vault's principal asset. This process relies on Operators to execute the swap and deliver the requested asset.
- ▶ **Deposit cap:** An upper limit on total Vault deposits has been introduced.

In the separate update, [PR #27](#) extends the protocol's Navi integration to support a single Navi account position across multiple markets. It introduces a registry of approved markets for each position and aggregates their values, updating the Vault's recorded position value only after all registered markets have been synchronized.

Code Assessment. The Volo Vault developers provided the source code of the Volo Vault contracts for the code review. The source code appears to be mostly original code written by the Volo Vault developers. It contains some documentation in the form of READMEs and documentation comments on functions and storage variables. To facilitate the Veridise security analysts' understanding of the code, the Volo Vault developers had previously provided a comprehensive overview of the project, detailing its core functions, system states, intended interaction patterns, and supported operations. For the current review, they provided a document outlining the changes introduced in this version.

The source code contained a test suite, which the Veridise security analysts noted covered many of the happy paths, but had relatively fewer tests to check for potential errors or flaws in the code.

Summary of Issues Detected. The security assessment uncovered 13 issues, with 4 classified as high severity or higher by the Veridise analysts.

Among the two critical-severity findings, [V-VVT-VUL-001](#) identifies missing binding between curator positions and vault asset slots, allowing an approved curator position value to overwrite the valuation of an unrelated vault asset; [V-VVT-VUL-002](#) shows that partial NAVI market updates can combine stale pre-operation values with post-operation values, allowing operator-caused losses to remain undetected.

The high-severity findings include [V-VVT-VUL-003](#), where withdraw-with-swap slippage protection is derived from execution-time prices rather than the user's request-time expectations, and [V-VVT-VUL-004](#), where curator loop operations change the vault's principal balance without refreshing its cached valuation.

The medium-severity findings include [V-VVT-VUL-005](#), where future-dated oracle values may be accepted as valid, [V-VVT-VUL-006](#), where NAVI positions are valued at their nominal net value without accounting for liquidation risk and expected recoverability, and [V-VVT-VUL-007](#),

where exposure to a NAVI reserve unsupported by the Volo oracle configuration may prevent the position from being valued and block vault operations.

The low-severity findings include [V-VVT-VUL-008](#), where a withdraw-with-swap request can be processed through the normal withdrawal path, causing the requested swap to be skipped and leaving its associated sidecar state unconsumed; and [V-VVT-VUL-009](#), where the deposit cap does not account for buffered pending deposits, allowing multiple accepted deposits to collectively exceed the configured limit.

The warning-severity findings include [V-VVT-VUL-010](#), where users can create withdraw-with-swap requests for unsupported target assets that cannot be priced or executed; [V-VVT-VUL-011](#), where an unbounded `slippage_bps` value can underflow during minimum-output calculation and leave a withdrawal request temporarily stuck; and [V-VVT-VUL-012](#), where the withdrawal minimum amount check is applied to the gross amount before fees rather than the net amount ultimately received by the user.

The informational finding, [V-VVT-VUL-013](#), documents several maintainability concerns, including misleading comments, redundant validation, and constants that are not derived from the protocol's shared rate-scaling value.

The report appendix additionally includes [V-VVT-APP-VUL-001](#), concerning stale NAVI position values after registering a new market, [V-VVT-APP-VUL-002](#), concerning inconsistent deposit-cancellation recipient semantics, and [V-VVT-APP-VUL-003](#), concerning mutable withdrawal and cancellation locking periods, all of which were classified by the developers as intended behavior.

The Volo Vault developers fixed 8 findings and acknowledged the remaining 5 findings without implementing further changes. Separately, the three appendix findings were confirmed by the developers as intended behavior.

Recommendations. After completing the security assessment, the analysts identified several recommendations to improve Volo Vault. Some of these were carried over from the previous report, as they remain relevant to the project.

Testing edge cases and system invariants. As a minimal step before deployment, it is recommended to add additional tests focused on edge cases and failure scenarios, rather than only expected user flows. Although the current test suite covers many important paths, including deposits, withdrawals, rewards, oracle updates, and general vault operations, it is less focused on the kinds of conditions under which subtle logic bugs often arise. In particular, the Veridise analysts recommend adding tests for receipt transfers before cancellation or execution, cancellation through different entry points leading to different refund recipients, slippage enforcement in zap-out flows, invalid user-supplied input values, withdrawal minimums after fees, curator updates affecting the wrong asset type, and oracle prices with future-dated timestamps.

The analysts also recommend strengthening the test suite by validating higher-level system invariants rather than only individual execution steps. For example, tests should confirm that balances and shares remain consistent after each action, that requests and receipts always transition to the correct final state, that transferred receipts do not create ambiguity around ownership or refund rights, and that an update to one asset cannot accidentally modify the value of another. Adding these kinds of checks would improve the protocol's ability to catch real security issues and logic regressions, rather than only demonstrating correctness in expected scenarios.

Clarify and formalize role intent and trust assumptions. The purpose, trust model, and expected behavior of all privileged roles—particularly Curators and Operators—should be clearly documented. This should articulate why certain roles are trusted, to what extent that trust applies, and how it is enforced. Where possible, these should be reflected directly in on-chain logic rather than relying on off-chain expectations.

Adopt more rigorous security assurance techniques. Given the protocol’s architectural complexity - multiple privileged roles, numerous Vault state transitions, multiple entry points and integrations with external protocols - it is recommended to complement traditional security review with more rigorous techniques such as model-based testing or model checking. These methods can help validate system-wide invariants and uncover edge cases that may be missed during manual review.

Improve code comments and technical documentation. Complex or non-obvious parts of the code-base should be supplemented with clearer and more precise comments. Particular attention should be given to asset valuation logic, especially the handling of token decimals. The adopted approach to decimals normalization should be explicitly described in both inline comments and external documentation.

Disclaimer. We hope that this report is informative but provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the system is secure in all dimensions. In no event shall Veridise or any of its employees be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the results reported here.

Table 2.1: Application Summary.

Name	Version	Type	Platform
Volo Vault	f8b817d	Move	Sui

Table 2.2: Engagement Summary.

Dates	Method	Consultants Engaged	Level of Effort
Mar. 25–Mar. 31, 2026	Manual & Tools	3	15 person-days

Table 2.3: Vulnerability Summary.

Name	Number	Acknowledged	Fixed
Critical-Severity Issues	2	2	2
High-Severity Issues	2	2	0
Medium-Severity Issues	3	3	2
Low-Severity Issues	2	2	2
Warning-Severity Issues	3	3	1
Informational-Severity Issues	1	1	1
TOTAL	13	13	8

Table 2.4: Category Breakdown.

Name	Number
Data Validation	6
Logic Error	5
Usability Issue	1
Maintainability	1



3.1 Security Assessment Methodology

The security assessment process consists of the following steps:

1. Gather an initial understanding of the protocol's business logic, users, and workflows by reviewing the provided documentation and consulting with the developers.
2. Identify all valuable assets in the protocol.
3. Identify the main workflows for managing these assets.
4. Identify the most significant security risks associated with these assets.
5. Systematically review the codebase for execution paths that could trigger the identified security risks, considering different assumptions.
6. Prioritize one finding over another by assigning a severity level to each.

3.2 Identified Security Risks

After the initial phase of the security assessment was completed, a list of potential security risks was generated. Security analysts used this list during the code review as a starting point to identify potential attack vectors. A few of these risks, when expressed as questions, include the following:

- ▶ Can the same pending deposit refund different recipients depending on who calls the operation?
- ▶ Does the oracle reject prices with timestamps in the future, or can not-yet-valid prices still be accepted?
- ▶ Can a valid curator position update be used to overwrite the value of an unrelated vault asset?
- ▶ Can a user submit an out-of-range slippage value that makes a withdraw-with-swap request impossible to execute later?
- ▶ Can privileged actors increase withdrawal or cancellation lock times after users have already deposited, making exit conditions worse retroactively?
- ▶ Does the withdraw-with-swap slippage check protect the user's expectation at request time, or only the price observed later at execution time?
- ▶ Can a user create a withdraw-with-swap request for an asset that the protocol cannot actually deliver, leaving the request stuck?
- ▶ Is it possible to exceed the vault's deposit cap by creating multiple pending deposits before earlier ones are executed?
- ▶ Can a withdraw-with-swap request be processed through a regular withdrawal path and bypass the swap-specific checks or cleanup?
- ▶ Are there any Sui-specific common implementation vulnerabilities like capability leaks, upgrade authority mistakes, shared-object overreach etc.

3.3 Scope

The scope of this security assessment is limited to a specific set of source files from the repository at commit f8b817d, as agreed upon with the Volo Vault developers:

- ▶ sources/operation.move
- ▶ sources/user_entry.move
- ▶ sources/volo_vault.move
- ▶ sources/requests/swap_request.move

In addition, the separate security review of [PR #27](#) was limited to the changes introduced by the pull request in the following source files at commit b151492:

- ▶ sources/adaptors/navi_adaptor.move
- ▶ sources/manage.move

3.4 Classification of Vulnerabilities

When Veridise security analysts discover a possible security vulnerability, they must estimate its severity by weighing its potential impact against the likelihood that a problem will arise.

The severity of a vulnerability is evaluated according to the Table 3.1.

Table 3.1: Severity Breakdown.

	Somewhat Bad	Bad	Very Bad	Protocol Breaking
Not Likely	Info	Warning	Low	Medium
Likely	Warning	Low	Medium	High
Very Likely	Low	Medium	High	Critical

The likelihood of a vulnerability is evaluated according to the Table 3.2.

Table 3.2: Likelihood Breakdown

Not Likely	A small set of users must make a specific mistake
Likely	Requires a complex series of steps by almost any user(s) - OR - Requires a small set of users to perform an action
Very Likely	Can be easily performed by almost anyone

The impact of a vulnerability is evaluated according to the Table 3.3:

Table 3.3: Impact Breakdown

Somewhat Bad	Inconveniences a small number of users and can be fixed by the user
Bad	Affects a large number of people and can be fixed by the user - OR - Affects a very small number of people and requires aid to fix
Very Bad	Affects a large number of people and requires aid to fix - OR - Disrupts the intended behavior of the protocol for a small group of users through no fault of their own
Protocol Breaking	Disrupts the intended behavior of the protocol for a large group of users through no fault of their own

4

Security Review Assumptions

4.1 Operational Assumptions

The Volo Vault integrates with several DeFi protocols, enabling Operators to earn profits for the protocol by temporarily borrowing Vault assets and utilizing them within these external platforms. While this integration adds significant functional complexity, it is important to note that not all of these DeFi protocols are thoroughly documented. Furthermore, the internal logic and security of these third-party protocols were explicitly out of scope for this security assessment. As a result, the assessment team had to rely on informed assumptions based on the behavior of similar protocols. The security posture of these integrated DeFi protocols remains uncertain.

Additionally, the Volo Vault relies on asset price data provided by Switchboard oracle price aggregators. The configuration, correctness, and security of these oracles were not within the scope of this review and were therefore not independently verified.

For the protocol to function as intended, Volo Vault depends on Operators to execute user requests and generate yield through flash-loan operations and other strategies. This operational logic resides entirely off-chain and was not reviewed as part of this assessment, nor was the security or trustworthiness of the Operators' execution environments evaluated.

This assessment was conducted under the following assumptions:

- ▶ Integrated DeFi protocols — specifically *Cetus*, *Momentum*, *Navi*, *ZO* and *SuiLend* — behave correctly and do not exhibit unexpected or non-standard behavior.
- ▶ Switchboard oracle aggregators are correctly configured, provide accurate price data, and operate securely.
- ▶ Admins, Operators, Managers and Curators are expected to act in accordance with the protocol's intended design and behave without malicious intent. However, in the case of privileged users, the system should ensure that even if their keys are compromised, they cannot steal funds from the Vault.
- ▶ The off-chain Operator logic is free from critical programmatic defects and is executed within a secure, trusted environment.

4.2 Privileged Roles

During the review, Veridise analysts assumed that the privileged users perform their responsibilities as intended. Protocol exploits relying on the following users acting outside of their privileged scope or intentionally abstaining from doing their duties are considered to be outside the scope of this security review.

- ▶ **Admin.** This user can enable, disable, and upgrade the Vault, set deposit and withdraw fees, set loss tolerance, assign Operator roles to users and freeze them; add, remove or change switchboard aggregators, as well as update various other contract states. There is a single Admin for the Vault.

- ▶ **Operators.** These types of users can perform privileged actions such as initiating flashloans, executing deposits and withdrawals, adding new asset types, and managing reward-related functionalities.
- ▶ **Managers.** These types of users can set and update parameters for individual vaults within the protocol.
- ▶ **Curators.** These types of users are third-party members that can submit curator positions and values, as well as loop-in or loop-out funds from the protocol.

Operational Recommendations. Highly-privileged operations should be operated by a multi-sig contract or decentralized governance system. These operations should be guarded by a time-lock to ensure there is enough time for incident response. Highly-privileged operations should be tested in example scenarios to ensure the role operators are available and ready to respond when necessary.

Full validation of operational security practices is beyond the scope of this review. Users of the protocol should ensure they are confident that the operators of privileged keys are following best practices such as:

- ▶ Never storing a protocol key in plain text, on a regularly used phone, laptop, or device, or relying on a custom solution for key management.
- ▶ Using separate keys for each separate function.
- ▶ Storing multi-sig keys in a diverse set of key management software/hardware services and geographic locations.
- ▶ Enabling 2FA for key management accounts. SMS should *not* be used for 2FA, nor should any account which uses SMS for 2FA. Authentication apps or hardware are preferred.
- ▶ Validating that no party has control over multiple multi-sig keys.
- ▶ Performing regularly scheduled key rotations for high-frequency operations.
- ▶ Securely storing physical, non-digital backups for critical keys.
- ▶ Actively monitoring for unexpected invocation of critical operations and/or deployed attack contracts.
- ▶ Regularly drilling responses to situations requiring emergency response such as pausing/unpausing.

5

Vulnerability Report

This section presents the vulnerabilities found during the security assessment. For each issue found, the type of the issue, its severity, location in the code base, and its current status (i.e., acknowledged, fixed, etc.) is specified. Table 5.1 summarizes the issues discovered:

Table 5.1: Summary of Discovered Vulnerabilities.

ID	Description	Severity	Status
V-VVT-VUL-001	Curator position value can be written to . . .	Critical	Fixed
V-VVT-VUL-002	Operator losses can be hidden through . . .	Critical	Fixed
V-VVT-VUL-003	Withdraw with swap operation does not . . .	High	Acknowledged
V-VVT-VUL-004	Curator loop flows leave principal asset . . .	High	Acknowledged
V-VVT-VUL-005	Oracle accepts future-dated prices as valid	Medium	Fixed
V-VVT-VUL-006	NAVI valuation ignores liquidation risk	Medium	Acknowledged
V-VVT-VUL-007	Unsupported NAVI reserve can prevent . . .	Medium	Fixed
V-VVT-VUL-008	Withdraw with swap requests can be . . .	Low	Fixed
V-VVT-VUL-009	Vault deposit cap can be exceeded due to . . .	Low	Fixed
V-VVT-VUL-010	Unsupported target asset can create an . . .	Warning	Acknowledged
V-VVT-VUL-011	Slippage bps greater than rate scaling can . . .	Warning	Fixed
V-VVT-VUL-012	Withdraw minimum amount check does . . .	Warning	Acknowledged
V-VVT-VUL-013	Maintainability concerns	Info	Fixed

5.1 Detailed Description of Issues

5.1.1 V-VVT-VUL-001: Curator position value can be written to an arbitrary asset slot

Severity	Critical	Commit	00cf6fb
Type	Data Validation	Status	Fixed
Location(s)	sources/curator_position.move:483-514		
Confirmed Fix At	volo-vault/pull/23		

Description The function `update_curator_position_value()` allows a caller to specify the `asset_type` that will be updated in the vault, but does not verify that the provided `curator_position_id` is actually associated with that asset.

The function correctly checks that the curator position belongs to the given vault, and that the curator value has been approved. However, it does not enforce any binding between the curator position and the asset slot being updated. The `asset_type` is taken directly from user input and passed into the vault, as can be seen in the snippet below.

```

1 public fun update_curator_position_value<PrincipalCoinType>(
2   self: &mut CuratorConfig,
3   vault: &mut Vault<PrincipalCoinType>,
4   asset_type: String,
5   curator_position_id: address,
6   oracle_config: &OracleConfig,
7   clock: &Clock,
8 ) {
9   let now = clock.timestamp_ms();
10  // veridise:: elided
11
12  let principal_based_position_value = self.position_value(curator_position_id);
13  let principal_price = vault_oracle::get_normalized_asset_price(
14    oracle_config,
15    clock,
16    type_name::with_defining_ids<PrincipalCoinType>().into_string(),
17  );
18  let position_value = vault_utils::mul_with_oracle_price(
19    principal_based_position_value,
20    principal_price,
21  );
22
23  vault.finish_update_asset_value(asset_type, position_value, now);
24 }

```

Snippet 5.1: Snippet from `update_curator_position_value()`

At the vault level, this results in a direct write into the selected asset slot. And because no validation is performed on `asset_type`, a caller can essentially supply any valid asset key within the vault and write the approved curator value into that slot.

Additionally, the function is public, meaning no special permissions are required to invoke it. Once a curator value has been approved, any caller can trigger this over-write and choose the target asset slot which is very dangerous.

Impact An approved curator value can overwrite the valuation of an unrelated asset in the same vault.

This can corrupt vault accounting, leading to incorrect total value, incorrect share pricing, and unfair deposits or withdrawals. Because asset values feed directly into NAV and accounting logic, the impact extends to all value dependent operations within the vault.

Recommendation Derive the curator `asset_type` internally from protocol maintained state rather than accepting it as caller input.

Developer Response The developers fixed this issue by separating curator assets from the generic DeFi asset flow and standardizing curator accounting around a single curator asset slot at index 0.

More specifically, `operation::add_new_defi_asset()` now rejects `CuratorPosition`, so curator positions can no longer be added through the generic asset entry path. Instead, a dedicated `add_curator_position_as_defi_asset()` function has been added for curator assets. This new function requires `CuratorConfig` as an input and checks that the provided `CuratorPosition` is actually paired with the vault it is being added to.

`update_curator_position_value()` has also been updated to not accept an arbitrary `asset_type` input and the function now requires that the provided `asset_type` match the vault designated curator asset key derived from `vault_utils::parse_key(0)`.

5.1.2 V-VVT-VUL-002: Operator losses can be hidden through partial NAVI market updates

Severity	Critical	Commit	b151492
Type	Logic Error	Status	Fixed
Location(s)	sources/ ▶ adaptors/navi_adaptor.move ▶ operation.move		
Confirmed Fix At	volo-vault/pull/27		

Description

Volo stores a separate cached value and update timestamp for the NaviAccountCap position in each registered NAVI market. It treats the complete position valuation as current when every market-specific value has the current timestamp.

However, the clock does not change within a single programmable transaction. An operator can:

1. Recalculate the NaviAccountCap position value in Markets A and B.
2. Start an operation and modify the position in Market A, reducing its real value.
3. Return the NaviAccountCap.
4. Recalculate only the position value in Market B.

Market A retains its pre-operation value, while Market B contains its post-operation value. Both entries still have the same timestamp, so Volo accepts and commits the mixed valuation.

The timestamp proves only that each market-specific value was calculated somewhere in the transaction. It does not prove that every value was recalculated after the latest NaviAccountCap change.

Impact

A paired operator can hide a reduction in the real value of a NAVI position. Operation accounting may report no loss, or less loss than actually occurred, without consuming the corresponding loss tolerance.

For example, the operator could withdraw collateral or increase debt in one market while Volo continues using that market's stale pre-operation value.

Recommendation

After recording the pre-operation vault value, invalidate all cached market-specific values for every NaviAccountCap taken from the vault. After it is returned, require every registered market value to be recalculated before the operation can complete.

Developers Response

The developers fixed the issue by implementing the recommendation.

5.1.3 V-VVT-VUL-003: Withdraw with swap operation does not implement real slippage protection

Severity	High	Commit	00cf6fb
Type	Logic Error	Status	Acknowledged
Location(s)	sources/user_entry.move:222-223		
Confirmed Fix At	N/A		

Description

The `slippage_bps` mechanism in the `withdraw_with_swap()` flow does not implement slippage protection in the usual user-facing sense.

When a user submits the `withdraw_with_swap()` request, the protocol stores only:

- ▶ the target asset type
- ▶ the recipient
- ▶ `slippage_bps`

It does not store the target-token quote, expected output amount, or minimum acceptable output amount at request time.

Later, when the operator executes the request, the protocol:

1. Computes the actual principal amount withdrawn,
2. Values the principal amount using the current oracle price of the principal asset,
3. Converts that value into the target asset using the current oracle price of the target asset,
4. Applies `slippage_bps` to that execution-time quote to derive `min_amount_out`.

As a result, the minimum output enforced by the protocol is anchored to the market state at execution time rather than the market state at request time.

Impact

Users are not protected against the primary risk that slippage controls are normally meant to address: deterioration in execution outcome after the request is submitted.

This can lead to materially worse settlement than the user's request-time expectation. For example:

- ▶ at request time, 100 SUI is worth 1 BTC,
- ▶ the user submits a request with 10% slippage tolerance,
- ▶ the user would reasonably expect to receive no less than 0.9 BTC,
- ▶ by execution time, 1 BTC is worth 200 SUI,
- ▶ the protocol now treats 100 SUI as worth 0.5 BTC,
- ▶ after applying 10% slippage, the enforced minimum becomes only 0.45 BTC.

The transaction still succeeds even though the user receives only half of what they would have considered acceptable when submitting the request.

This weakens user protection and gives operators broad discretion to execute against significantly worse market conditions without violating the on-chain slippage check.

Recommendation

Anchor slippage protection to the request-time quote, not the execution-time quote.

At minimum, when the user submits `withdraw_with_swap`, the protocol should store either:

- ▶ the request-time quoted target amount, or
- ▶ the derived `min_amount_out` directly

Then, during execution, the protocol should validate the delivered target amount against that stored request-time floor, optionally scaled only for the actual principal amount withdrawn if the actual withdrawal amount can legitimately differ from the user's estimate.

A safer model is:

1. At request time, compute and store:
 - ▶ $\text{theoretical_target_amount} = \text{expected_principal_amount} * \text{request_principal_price} / \text{request_target_price}$
2. At execution time:
 - ▶ require that the final delivered target amount is no less than the stored slippage-adjusted minimum.

This would make `slippage_bps` reflect the user's actual expectation at request time and provide real protection against adverse price movement before execution.

Developers Response

The developers acknowledged the issue and do not plan on providing a fix, with the quote:

We discussed and confirmed that this is an intended design. Unlike a traditional DEX swap, we use pre-configured slippage bps instead of a minimum received amount. This is because there may be a price difference between the request time and the execution time. A minimum-received strategy is very likely to fail and may not reflect market price changes.

We do acknowledge that a malicious operator could exploit this mechanism. This will be disclosed to users, and we have built a rigorous management flow for operators.

5.1.4 V-VVT-VUL-004: Curator loop flows leave principal asset value stale

Severity	High	Commit	00cf6fb
Type	Logic Error	Status	Acknowledged
Location(s)	sources/curator_position.move		
Confirmed Fix At	N/A		

Description

The curator loop flows mutate the vault's principal balance without refreshing the cached USD value of that principal.

Specifically:

- ▶ `loop_in_curator_position_with_claimable_balance` and `loop_in_curator_position` reduce principal exposure by calling `borrow_free_principal()`
- ▶ `loop_out_curator_position_internal()` increases principal exposure by calling `return_free_principal()`

Those helpers only move `free_principal`. They do not call `update_free_principal_value()`, so the principal asset's cached USD value in the vault can remain stale after the balance change.

That matters because the vault's accounting uses cached asset values. The `get_total_usd_value()` does not recompute balances; it only reads cached USD values of all assets and checks that their timestamps are fresh enough. If a curator loop changes `free_principal` and no subsequent principal value refresh occurs first, later share-pricing logic can operate on an outdated principal asset value.

Impact

This can cause deposits, withdrawals, and other share-ratio-dependent flows to use an incorrect total vault value.

If principal was looped out to a curator but the cached principal asset value was not reduced yet, the vault can appear richer than it is. That can misprice shares and distort user entry or exit terms. If principal was looped back in but the cached principal asset value was not increased yet, the vault can appear poorer than it is, again causing incorrect share pricing.

The practical effect is accounting drift during the window between the curator loop and the next manual principal value refresh. Any user flow that relies on total USD value during that window may settle against stale principal asset value.

Recommendation

Refresh principal valuation immediately whenever curator loop flows move principal into or out of `free_principal`.

The cleanest fix is to ensure:

- ▶ `loop_in_curator_position_with_claimable_balance()`
- ▶ `loop_in_curator_position()`
- ▶ `loop_out_curator_position_internal()`

all update principal asset value as part of the same flow.

Developers Response

The developers acknowledged the issue with the following comment:

This is an intended design. Indeed, it is more consistent if the position value is updated after looping out, but it would increase the code complexity. We decided to let the curator update the value manually.

5.1.5 V-VVT-VUL-005: Oracle accepts future-dated prices as valid

Severity	Medium	Commit	00cf6fb
Type	Data Validation	Status	Fixed
Location(s)	sources/oracle.move:261-263		
Confirmed Fix At	volo-vault/pull/20, 0a31d79		

Description

The oracle logic incorrectly accepts aggregator prices whose timestamp lies in the future.

In `sources/oracle.move:255`, the code reads the aggregator's `max_timestamp_ms()` and only enforces freshness if the reported timestamp is less than or equal to the current block time:

```

1 if (now >= max_timestamp) {
2   assert!(now - max_timestamp < config.update_interval, ERR_PRICE_NOT_UPDATED);
3 };

```

If `max_timestamp > now`, no validation is applied and the price is accepted unconditionally.

Impact

The protocol can accept oracle values that are not valid for the current block time.

If a future-dated aggregator result is used, the vault may:

- ▶ update cached asset prices using data that should not yet be considered live,
- ▶ use those prices in valuation-sensitive logic such as total USD value, share ratio, withdrawals, deposits, and strategy accounting,
- ▶ make state transitions based on temporally invalid market data.

Recommendation

Reject oracle updates whose timestamp is in the future.

A straightforward fix is to require:

```

1 assert!(max_timestamp <= now, ...);
2 assert!(now - max_timestamp < config.update_interval, ERR_PRICE_NOT_UPDATED);

```

Developers Response

The developers now check that `max_timestamp` does not exceed the current chain timestamp by more than a bounded tolerance.

5.1.6 V-VVT-VUL-006: NAVI valuation ignores liquidation risk

Severity	Medium	Commit	b151492
Type	Logic Error	Status	Acknowledged
Location(s)	sources/adaptors/navi_adaptor.move:216-260		
Confirmed Fix At	N/A		

Description Volo values the position controlled by a NaviAccountCap separately in each whitelisted NAVI market as $\max(\text{supplied value} - \text{borrowed value}, 0)$, then sums the market-specific values.

Impact The vault may value unhealthy NAVI positions above the amount recoverable after liquidation. Multi-market support increases this risk because one NaviAccountCap can have independently liquidatable positions in several markets. Multiple market-specific positions may be liquidated during the same period, causing liquidation bonuses and losses to materialize across several components of the aggregated value.

This can overstate the vault share price, overpay withdrawals, under-mint deposit shares, and allow strategy loss checks to use an inflated post-operation value. The resulting losses are borne by vault shareholders.

Recommendation Value NAVI positions based on their recoverable value rather than simple collateral minus debt. Assess the health of the position in each market independently and do not include an unhealthy position at its full net value.

If a NAVI market position is unhealthy, do not use its nominal net value for share pricing. Either value it conservatively after accounting for expected liquidation losses, or pause deposit and withdrawal execution while the operator repairs or unwinds the position.

Developer Response The developers have acknowledged the issue, with the quote:

This is an intended design. As a strategy vault, it always contains potential risk. It occurs in both single and multiple markets. Even though there is no liquidation risk, it also faces bad debt risk, protocol operation risk, and price depegging risk. We will disclose the underlying assets and our strategy to users.

5.1.7 V-VVT-VUL-007: Unsupported NAVI reserve can prevent market valuation

Severity	Medium	Commit	b151492
Type	Data Validation	Status	Fixed
Location(s)	sources/adaptors/navi_adaptor.move		
Confirmed Fix At	volo-vault/pull/27		

Description Volo prices every NAVI reserve in which the vault has a nonzero supply or borrow balance. Reserves with zero exposure are skipped.

NAVI and Volo maintain independent oracle configurations, but market registration does not verify that Volo can price every NAVI reserve. If the vault gains exposure to a reserve without a Volo price feed, `get_asset_price` aborts with `ERR_AGGREGATOR_NOT_FOUND`.

Impact The affected NAVI position cannot be refreshed. Consequently, operations requiring a fresh vault value, including deposit execution, withdrawal execution, and strategy settlement, can no longer complete.

The market also cannot be removed normally because removal requires a freshly synchronized value of zero. This creates a vault-level denial of service until the missing oracle is added or the position is repaired through another privileged path.

A related issue is that NAVI markets can gain new reserves after a `NaviAccountCap` has been registered with Volo Vault. If a new reserve lacks a Volo oracle, even a dust-sized supply or borrow position in that reserve causes the entire `NaviAccountCap` valuation to abort, blocking vault operations that depend on valuation.

Developer Response The developers updated NAVI position valuation so that an unsupported reserve is skipped when it contains supply but no borrow balance, preventing a missing price feed from blocking valuation while only understating the position’s value. If the unsupported reserve contains debt, valuation still aborts to avoid ignoring a liability and overstating the position’s value.

A follow-up fix also requires every reserve in a NAVI market to have zero supply and zero borrow when the market is registered with the vault. This prevents an operator from onboarding a pre-existing NAVI position containing unsupported debt that would subsequently block valuation.

5.1.8 V-VVT-VUL-008: Withdraw with swap requests can be executed as normal withdrawals

Severity	Low	Commit	00cf6fb
Type	Data Validation	Status	Fixed
Location(s)	sources/operation.move:451		
Confirmed Fix At	volo-vault/pull/20		

Description The `withdraw_with_swap()` flow creates a standard `WithdrawRequest` and stores the swap specific intent separately in a `WithdrawSwapRequest` sidecar, keyed by the same `request_id`. However, the legacy execution paths (`execute_withdraw()` and `batch_execute_withdraw()`) do not check whether a withdraw request is associated with a swap before executing it. As a result, a swap tagged request can still be executed through the legacy withdraw workflow. In such a scenario, a malicious operator could directly transfer the principal asset to the user, ignoring the swap intent entirely. The main issue stems from the fact that to the legacy withdrawal functions the withdraw-with-swap request remains indistinguishable from a normal withdraw request.

Moreover, this issue is not necessarily limited to malicious operator behavior. Currently, the `withdraw_with_swap()` function does not emit any explicit event indicating that a swap based withdrawal was requested. Instead, it relies on the underlying `request_withdraw()` call, which emits a standard `WithdrawRequested` event. Therefore, from a canonical event surface, to an external observer the request would appear identical to a normal withdrawal.

```

1 public fun withdraw_with_swap<PrincipalCoinType, TargetAssetType>(
2     vault: &mut Vault<PrincipalCoinType>,
3     shares: u256,
4     expected_amount: u64,
5     receipt: &mut Receipt,
6     clock: &Clock,
7     slippage_bps: u64,
8     ctx: &mut TxContext,
9 ): u64 {
10     vault.assert_vault_receipt_matched(receipt);
11     assert!(
12         vault.check_locking_time_for_withdraw(receipt.receipt_id(), clock),
13         ERR_WITHDRAW_LOCKED,
14     );
15     assert!(shares > 0, ERR_INVALID_AMOUNT);
16
17     let recipient = ctx.sender();
18     let target_asset_type = type_name::with_defining_ids<TargetAssetType>().
19         into_string();
20
21     let request_id = vault.request_withdraw(
22         clock,
23         receipt.receipt_id(),
24         shares,
25         expected_amount,
26         recipient,
27     );
28     let swap_request = swap_request::new_withdraw_swap_request(

```

```

29     vault.vault_id(),
30     request_id,
31     recipient,
32     target_asset_type,
33     slippage_bps,
34 );
35 let withdraw_swap_requests = swap_request::withdraw_swap_requests_mut(vault);
36 withdraw_swap_requests.add(request_id, swap_request);
37
38     // Missing WithdrawWithSwap event
39     request_id
40 }

```

Unless an operator explicitly inspects the `withdraw_swap_requests` stored in the vault, there is no clear signal that swap specific handling is required. This makes it possible for an honest operator to mistakenly process a request with swap intent through the legacy execution path.

Impact A `withdraw_with_swap` request may be settled in the wrong asset. Instead of receiving the requested target asset, the user may receive the principal asset directly, bypassing the swap execution and any associated constraints such as minimum output or slippage limits.

Additionally, the swap sidecar `WithdrawSwapRequest` will not be consumed when the legacy path is used, leaving stale state behind.

Recommendation Ensure that withdraw-with-swap requests cannot be executed through the legacy withdraw paths `execute_withdraw()` or `batch_execute_withdraw()`. Reject any `request_id` that has an associated swap sidecar, and enforce swap handling happens through the dedicated execution path.

Developer Response The developers provided the following response:

By design, the swap in `withdraw_with_swap` is the operator's discretion, not an obligation: when the target asset's price/slippage is unfavorable, the operator can skip the swap and return the principal directly as a safe fallback - the user gets equivalent value, so there's no loss. So letting a swap request settle via the legacy `execute_withdraw` path is intentional, not a bug.

The issue's suggested fix (rejecting swap-tagged requests on the legacy path) would actually break that fallback - the request would be unexecutable on both paths whenever the market is bad. The only real defect was that the legacy path left the swap sidecar behind (orphan state), so the correct fix is simply to clean it up with `try_delete_withdraw_swap_request`, not to restrict the operator.

5.1.9 V-VVT-VUL-009: Vault deposit cap can be exceeded due to buffered pending deposits

Severity	Low	Commit	00cf6fb
Type	Logic Error	Status	Fixed
Location(s)	sources/vault_config.move:52		
Confirmed Fix At	volo-vault/pull/20		

Description The deposit cap enforcement only considers the vault's current `free_principal` when validating new deposits, and does not account for deposits that have already been accepted but are still buffered.

When a user submits a deposit, the protocol checks that the current `free_principal` plus the deposit amount does not exceed the configured `deposit_cap`.

```
1 assert!(  
2 (vault.free_principal() as u128) + (deposit_amount as u128) <= (config.deposit_cap as  
   u128),  
   ERR_DEPOSIT_CAP_EXCEEDED,  
3 );
```

Snippet 5.2: Snippet from `assert_deposit_allowed()`

However, accepted deposits are not immediately added to `free_principal`. Instead, they are stored in `request_buffer.deposit_coin_buffer` and only later moved into `free_principal` during execution. This creates a gap between validation and accounting. At validation time, only the already executed principal is considered, while buffered deposits despite already being under protocol custody are ignored. As a result, multiple deposits can independently pass the cap check while sitting in the buffer, and later be executed together.

For example, if the deposit cap is 100 and `free_principal` is 0, two users can each deposit 80. Both the deposits pass the current deposit cap validation because buffered deposits are not considered. When executed, both deposits then get added to `free_principal`, resulting in a total of 160 which exceeds the deposit cap.

Impact The deposit cap does not act as a true upper bound on total deposited principal, as intended. Multiple buffered deposits can bypass the cap and be executed later, causing the vault to exceed its configured limit. This makes any assumptions or safeguards based on the deposit cap unreliable.

Recommendation Enforce the deposit cap at execution time by checking it before buffered deposits are added to `free_principal`.

Alternatively, include buffered deposits in deposit cap calculations so that pending deposits are accounted for during validation.

Developer Response The developers provided the following statement:

We have designed to remove the feature completely. Therefore, the feature of deposit limit will no longer be present.

5.1.10 V-VVT-VUL-010: Unsupported target asset can create an unexecutable withdraw-with-swap request

Severity	Warning	Commit	00cf6fb
Type	Data Validation	Status	Acknowledged
Location(s)	sources/user_entry.move:216-254		
Confirmed Fix At	N/A		

Description The `withdraw_with_swap()` method allows users to create a swap withdrawal request for any `TargetAssetType`, without validating whether that asset is supported by the oracle.

At request creation, the target asset is recorded only as a type string. There is no check to ensure that the asset has a corresponding oracle price feed.

```
1 let target_asset_type = type_name::with_defining_ids().into_string();
```

Snippet 5.3: Snippet from `withdraw_with_swap()` in `user_entry.move`

As a result, requests can be accepted for assets that cannot be priced. However, these requests cannot be fulfilled, because execution requires computing the minimum acceptable output using oracle prices for both the principal and target assets. This in turn requires the target asset to have a configured oracle aggregator. If no such aggregator exists, execution aborts.

```
1 let principal_price = vault_oracle::get_normalized_asset_price(...);
2 let target_price = vault_oracle::get_normalized_asset_price(...);
```

Snippet 5.4: Snippet from `execute_withdraw_with_swap()` in `operation.move`

Therefore, the system allows the creation of requests that are not executable under the current configuration. This can be a source of inconvenience for the user, as they may not be able to immediately triangulate why their request remains unfulfilled.

Impact Users can create swap requests that appear valid but cannot be executed. These requests remain pending until either the user cancels them or oracle support is added for the target asset.

Since cancellation is subject to a lock period, users may be unable to recover or reroute their funds for a non-trivial amount of time, leading to a negative user experience.

Recommendation Validate oracle support for `TargetAssetType` during request creation, and reject unsupported assets.

Developer Response The developers have acknowledged the issue and do not plan on providing a fix, with the quote:

We will cancel the request in this case.

5.1.11 V-VVT-VUL-011: Slippage bps greater than rate scaling can brick zap-out execution

Severity	Warning	Commit	00cf6fb
Type	Data Validation	Status	Fixed
Location(s)	sources/user_entry.move:222-223		
Confirmed Fix At	volo-vault/pull/20		

Description

The `withdraw_with_swap()` flow does not validate that the user-supplied `slippage_bps` is less than `RATE_SCALING`.

When a request is created, the provided `slippage_bps` is stored without bounds checking. Later, during execution, the protocol computes:

```
min_amount_out = expected_target_amount * ((RATE_SCALING - slippage_bps) as u256) / (
RATE_SCALING as u256);
```

If `slippage_bps > RATE_SCALING`, the subtraction `RATE_SCALING - slippage_bps` can underflow and cause execution to abort.

Impact

A user can create a malformed `withdraw_with_swap()` request that cannot be executed successfully.

In that case:

- ▶ the withdrawal request remains pending,
- ▶ the receipt's pending withdrawal state remains non-zero,
- ▶ the operator cannot complete the zap-out execution path,
- ▶ the user must manually cancel the request to restore normal receipt state.

Because cancellation is itself time-locked, the user may also need to wait for the cancellation timeout before recovering from the mistake.

This is primarily a denial-of-service against the caller's own request rather than a protocol-wide vulnerability. The impact is therefore limited, but it still creates avoidable stuck state and unnecessary operational friction.

Recommendation

Validate `slippage_bps` to be less than `RATE_SCALING` when the request is created.

Developers Response

The developers fixed the issue by implementing the recommendation.

5.1.12 V-VVT-VUL-012: Withdraw minimum amount check does not protect net amount received

Severity	Warning	Commit	00cf6fb
Type	Usability Issue	Status	Acknowledged
Location(s)	sources/volo_vault.move:1222-1223		
Confirmed Fix At	N/A		

Description

The withdrawal slippage check in `volo_vault.execute_withdraw()` is performed against the gross withdrawal amount, while the user ultimately receives the net amount after the protocol fee is deducted.

In `volo_vault.execute_withdraw()`, the protocol:

1. computes `amount_to_withdraw` from the user's shares,
2. checks that `amount_to_withdraw >= expected_amount`,
3. deducts the withdrawal fee from `amount_to_withdraw`,
4. transfers or credits only the remaining amount to the user.

As a result, users receive less than they specified in the `expected_amount`.

Impact

The `expected_amount` parameter is not enforced on the amount the user actually receives. A request can satisfy the slippage check and still settle below the user's stated expectation once the withdrawal fee is applied.

Recommendation

Apply the user's minimum-output check to the net amount after fees, not the gross amount before fees.

A straightforward fix would be:

1. compute `amount_to_withdraw`,
2. compute the withdrawal fee,
3. derive the net amount the user will actually receive,
4. enforce `net_amount >= expected_amount`.

If the intended design is instead that `expected_amount` represents a pre-fee amount, this should be made explicit in both code semantics and user-facing documentation. Even in that case, renaming the parameter to something like `expected_gross_amount` would reduce ambiguity and help prevent integration mistakes.

Developers Response

The developers acknowledged the issue but indicated that they do not plan to provide a fix.

5.1.13 V-VVT-VUL-013: Maintainability concerns

Severity	Info	Commit	00cf6fb
Type	Maintainability	Status	Fixed
Location(s)	sources/ ▶ manage.move:203 ▶ requests/deposit_request.move:9-10 ▶ user_entry.move:192-193 ▶ volo_vault.move:32-33, 1792-1793		
Confirmed Fix At	volo-vault/pull/20		

Description The following maintainability issues were identified throughout the codebase.

Misleading or Incorrect comments:

- manage.move @ L203:**
The comment indicating that an operator can only manage one vault seems to be incorrect. An operator can manage multiple vaults but they cannot be added to the same vault twice.
- deposit_request.move @ L9-10:** The second half of the comment for the recipient regarding the "with_lock" flag is ambiguous.
- volo_vault.move @ L1792-1793:** The comment on vault.retrieve_deposit_withdraw_fee() denotes that the function can only be called by admins, however, it can also be called by operators via manage.retrieve_deposit_withdraw_fee_by_operator().

Miscellaneous:

- user_entry.move @ L192-193:** The vault.assert_normal() check in user_entry.cancel_withdraw() is redundant as the same check takes place in vault.cancel_withdraw().
- volo_vault.move @ L32-33:**
Rates are currently set as pure numbers whereas, ideally, they should be expressed through RATE_SCALING. If the latter value changes, all other values that depend on it will become invalid without manual adjustment.
So, for example:

```
1 BPS = RATE_SCALING / 100
2
3 DEPOSIT_FEE_RATE = BPS / 10
4
5 MAX_DEPOSIT_FEE_RATE = 5 * BPS // etc.
```

Impact The mentioned maintainability concerns may degrade code quality and negatively impact long term maintainability.

Recommendation It is recommended to address the mentioned maintainability concerns.

Developers Response The developers addressed the identified maintainability concerns.



A.1 Intended Behavior: Non-Issues of Note

A.1.1 V-VVT-APP-VUL-001: Adding a NAVI market leaves a stale position value

Severity	High	Commit	b151492
Type	Logic Error	Status	Intended Behavior
Location(s)	sources/adaptors/navi_adaptor.move		

Description Volo values a NaviAccountCap by aggregating its positions across the NAVI markets currently registered for that vault asset. The resulting value is cached and considered current for the transaction in which it was calculated.

An operator can first refresh the NAVI position and then register another market during the same transaction. The newly registered market may already contain assets controlled by the NaviAccountCap, but registering it does not invalidate the aggregate value calculated before the market was added.

Because all actions in the transaction share the same timestamp, Volo continues to treat the earlier aggregate as current. The operator can therefore execute a pending deposit before refreshing the new market. The deposit is priced using a vault value that excludes the assets held in that market, causing the depositor to receive more shares than the complete vault value would justify.

Impact A malicious operator can execute a colluding deposit at a stale, understated share price immediately after registering a valuable NAVI market. This over-mints shares and transfers value from existing shareholders to the new depositor.

The loss scales with the value held in the newly registered market and can be realized by later withdrawing the over-minted shares.

Consider the following attack scenario:

Preconditions

- ▶ User A owns all existing vault shares.
- ▶ The vault reports \$100 of free principal and has 100 shares.
- ▶ Its NaviAccountCap controls an additional vault-owned \$900 position in Market B.
- ▶ Market A is registered, but Market B is not yet registered.
- ▶ User B has a pending \$100 deposit.
- ▶ User B is controlled by or colludes with the operator.
- ▶ Fees are ignored for simplicity.

The vault’s reported value is \$100, but its complete economic value is \$1000.

Attack Scenario

1. In one PTB at timestamp t , the operator refreshes the free-principal value and the registered NAVI markets.
2. Volo caches a total vault value of \$100 at timestamp t .
3. The operator registers Market B, which contains the additional \$900 NAVI position.
4. Market B is initialized as unsynchronized, but the parent NaviAccountCap value remains marked current at timestamp t .
5. The operator executes User B's \$100 deposit before refreshing Market B.
6. Volo calculates the share price as $\$100 / 100 \text{ shares} = \1 .
7. User B receives 100 shares.
8. The operator then refreshes Market B, and the vault's complete value becomes \$1,100.
9. The vault now has 200 shares worth \$5.50 each.

If Market B had been included before the deposit, the correct pre-deposit share price would have been $\$1,000 / 100 \text{ shares} = \10 . User B should therefore have received only 10 shares.

Instead, User B's 100 shares are worth \$500 after synchronization. User A's claim falls from \$1000 to \$550, transferring approximately \$450 of value to User B.

Recommendation Invalidate the parent NaviAccountCap valuation whenever a market is added or removed. Do not allow the aggregate value to become current again until every whitelisted market has been refreshed after the registry change.

Developer Response The developers confirmed that this is an Intended behavior with the quote:

In step 3 of the attack scenarios, the operator registers Market B, which contains the additional \$900 NAVI position. This assumes that someone actually donates funds to the vault, but the operator doesn't allocate the funds to users "fairly". Also, if an operator allocates funds to an unregistered market by mistake, the loss tolerance check will not be passed. This is equal to sending funds out from the smart contract. In a very rare case, someone really donates funds to the position, we believe the operator should have the right to decide how the donated funds are allocated. Anyway, users won't lose any of their money.

A.1.2 V-VVT-APP-VUL-002: Deposit cancellation routes funds to different recipients depending on the entry point

Severity	Medium	Commit	00cf6fb
Type	Logic Error	Status	Intended Behavior
Location(s)	sources/user_entry.move:95		

Description

The deposit cancellation flow handles the request recipient inconsistently depending on which entry point is used.

When a deposit request is created, the request stores a recipient field. However, if a user later cancels that deposit through the receipt-based path, the cancelled funds are returned to the caller of the cancellation rather than to the original request recipient.

Specifically:

- ▶ In the user-facing cancellation flow, the `ctx.sender()` is used and the stored `deposit_request.recipient()` is not enforced.
- ▶ In the operator cancellation flow, the operator can cancel the same pending deposit through a separate path, but the funds are routed using the original request recipient instead.

As a result, the same pending deposit request can resolve to different recipients depending on which cancellation entry point is used.

Impact

Pending deposit ownership and refund semantics become ambiguous once receipts are transferable.

A deposit may be initiated by one address, while the corresponding receipt is later transferred to another address before the pending deposit is executed or cancelled. At that point, the protocol has two conflicting notions of ownership:

- ▶ the request metadata still points to the original deposit recipient,
- ▶ the receipt and its `pending_deposit_balance` are now held by a new owner, who reasonably expects to control the pending position.

Because the cancellation outcome depends on which path is used, the same pending deposit can resolve to different recipients:

- ▶ if the current receipt holder cancels through the user path, the funds are returned to that holder,
- ▶ if the operator cancels through the operator path, the funds are returned to the original request recipient.

This creates two concrete harmful scenarios:

1. **Collusion or malicious operator behavior** Alice deposits and receives a receipt, then transfers or sells that receipt to Bob. Bob expects that the pending deposit position represented by the receipt belongs to him. Before the deposit is executed, a malicious or colluding operator cancels the pending deposit through the operator path. Because that path routes funds using the original request recipient, the refunded assets go back to Alice instead of Bob. Bob loses the value he believed he acquired through the receipt transfer.

2. **Non-malicious operational cancellation** Alice deposits and transfers the receipt to Bob before execution. Later, the pending deposit needs to be unwound for ordinary operational reasons, such as changed market conditions, paused strategy execution, stale requests, or support-driven cancellation. The operator uses the normal operator cancellation flow, relying on the request's stored recipient as the apparent source of truth. The protocol refunds Alice rather than Bob, even though Bob is the current receipt holder and the pending deposit was economically represented on his receipt. In this scenario, Bob is harmed even without malicious intent, purely because the protocol uses inconsistent ownership sources.

In both cases, the protocol can cause the economic value of a pending deposit to diverge from the ownership implied by the receipt.

Recommendation

Unify the recipient model for deposit cancellation across all entry points.

Choose one of the following designs and enforce it consistently:

- ▶ Request-recipient model: always return cancelled deposit funds to `deposit_request.recipient()`, regardless of who submits the cancellation transaction.
- ▶ Receipt-owner model: always return cancelled deposit funds to the current valid receipt holder, and remove or clearly de-emphasize the request recipient field for deposit cancellation semantics.

Developers Response

The developers state that the described issue is an intended behavior of the protocol, with the quote:

For OP cancellation, the funds are intentionally sent back to the original depositor. This is designed for frontend EOA/wallet users. It is difficult and unnecessary to track the object owner of the receipt, especially when the receipt is held by a shared object.

However, we understand that some users may assume the cancelled funds will go to the new owner after a transfer. We identify this as a user misunderstanding rather than a protocol-level exploitable bug. We will highlight this mechanism in the technical documentation.

A.1.3 V-VVT-APP-VUL-003: Locking periods are mutable and can unfairly delay user exits

Severity	Low	Commit	00cf6fb
Type	Usability Issue	Status	Intended Behavior
Location(s)	sources/manage.move:108-122		

Description

The vault allows privileged actors to change the withdrawal and request-cancellation locking periods after users have already deposited funds.

In particular, the vault exposes mutable parameters for:

- ▶ `locking_time_for_withdraw`
- ▶ `locking_time_for_cancel_request`

Impact

Users are exposed to unfair and unpredictable changes in exit conditions after funds are deposited. A privileged party can increase withdrawal or cancellation locking periods after deposits have already been made, causing users to:

- ▶ wait substantially longer than expected before withdrawing,
- ▶ lose the ability to cancel pending actions in the timeframe they assumed,
- ▶ face a materially different liquidity profile than the one under which they entered the vault.

Recommendation

Introduce a hard upper bound for both withdrawal and cancellation locking times.

At minimum:

- ▶ define immutable protocol-level maximums for `locking_time_for_withdraw` and `locking_time_for_cancel_request`,
- ▶ enforce those maximums in all setter functions.

Developers Response

The developers stated the described issue is an intended behavior of the protocol, with the quote:

This design is slightly in favor of the protocol. But we believe this is acceptable as the cap is held by the admin, not the operator. This could give the admin a bit more flexibility in some special cases